Are Self-Evolving Agent Skills Learning Skills?

Anonymous submission

Abstract

We introduce a lifecycle audit for determining whether score gains from self-evolving agent skills represent retained skill improvement. A post-revision score alone is insufficient because optimizer variation, validation selection, execution randomness, and verifier choice can improve the endpoint without creating a durable artifact effect. We combine repeated evaluations, controlled task variations, and lifecycle interventions to examine whether improvements from skill evolution persist and transfer, and to trace their origins to feedback, revision, selection, skill content, and evaluation. The audit separates five evidential claims: selected gain, retained artifact effect, bounded transfer, evolution-pathway dependence, and measurement dependence. This claim-grounded framework lets self-evolving systems report what their experiments establish without equating a better benchmark score with reusable skill learning.

Introduction

Persistent agent skills offer an appealing route to continual improvement. Instead of updating model parameters, an agent can revise an external artifact that stores reusable instructions, scripts, references, templates, or other supporting resources. Recent systems generate, repair, select, and retain such artifacts from execution traces and evaluator feedback, often reporting higher post-evolution benchmark scores as evidence that the agent has learned a better skill (Li and Hu 2026; Liu et al. 2026b; Yang et al. 2026; Xia et al. 2026).

A higher score is not, by itself, an identified skill effect. The selected run may be unusually favorable; the optimizer may produce variable candidates; a validation gate may retain a narrow patch; repeated execution may eventually succeed without a durable behavioral change; and a verifier may reward only part of the intended behavior. These possibilities do not imply that self-evolution is ineffective. They imply that an endpoint score alone cannot locate where the improvement resides.

We therefore ask two connected questions. When self-evolution raises the benchmark score, does the revised skill itself retain a behavioral improvement under fresh deployment and controlled task variation? Regardless of whether it does, which parts of the evolution-and-evaluation lifecycle produced the observed gain?

We organize the evidence as a diagnosis of utility, behavioral scope, and gain attribution. Standard-task evaluations of SkillEvolver, SPARK, and CoEvoSkills first establish the observed utility of completed artifacts across two runs. Controlled task changes then reveal different scopes of usefulness, and artifact and trace inspection examines the decisions preserved during reuse. This motivates the next diagnostic step: locating an improvement in the artifact's evolution history. Complementary studies of SkillRevise, SkillHone, and SkillOpt use retention tests and lifecycle interventions to examine trace-conditioned revision, history-based iterative honing, and validation-gated editing. Each study addresses a specific part of the identification question.

Our audit follows the lifecycle from feedback to revision, selection or adoption, frozen artifact deployment, and measurement. We separate historical success from fresh retention, final-skill utility from actual parent-to-final revision effects, and same-task retention from controlled transfer. We then intervene on feedback, optimizer sampling, gate provenance, artifact components, and verifier choice. A short information-integrity check bounds direct answer or oracle exposure as an alternative explanation; it is not the paper's organizing hypothesis.

The resulting picture is deliberately more qualified than either *self-evolution works* or *self-evolution only exploits the evaluator*. Revised artifacts can cause real behavioral improvements, but the demonstrated effects are often narrow. Single-run or selected gains need not survive fresh deployment. Useful traces and feedback must be converted into an effective edit, adopted into the persistent artifact, and retained behaviorally. Finally, a better score can reflect compatibility with a measurement surface rather than broader task behavior.

Our central contribution is a claim-grounded lifecycle audit: each reported gain is matched to the intervention required to support it and to the strongest conclusion that intervention permits. We make three contributions:

- **Identification framework.** We separate a reported gain from retention, behavioral scope, evolution-and-selection pathway, and measurement dependence.

This is an anonymized submission for review purposes only. Distribution, citation, or public sharing of this manuscript is strictly prohibited. Copyright and publication details will appear in the final version if accepted.

自进化代理的技能是否在学习技能?

匿名提交

摘要

我们介绍了一种生命周期审计方法，用于判断自进化代理技能带来的分数提升是否代表持久的技能改进。仅凭修订后的分数是不够的，因为优化器变化、验证选择、执行随机性和验证器选择都可能在不创建持久性效果的情况下提高终点分数。我们结合重复评估、受控任务变化和生命周期干预，来检验技能进化带来的改进是否持久且可迁移，并追溯到反馈、修订、选择、技能内容和评估的来源。该审计将证据声明分为五类：选择增益、持久性效果、有界迁移、进化路径依赖和测量依赖。这个基于声明的框架允许自进化系统报告其实验结果，而不将更好的基准分数等同于可重用的技能学习。

引言

持久的代理技能为持续改进提供了一条有吸引力的途径。代理无需更新模型参数，而是可以修订存储可重用指令、脚本、参考资料、模板或其他支持资源的的外部工件。最近的系统从执行轨迹和评估器反馈中生成、修复、选择和保留此类工件，通常报告更高的进化后基准分数，以此作为代理学习了更好技能的证据（Li和Hu 2026；Liu等人 2026b；Yang等人 2026；Xia等人 2026）。

更高的分数本身并不能确定是某种技能效果。选定的运行可能特别有利；优化器可能产生不同的候选方案；验证门可能保留一个狭窄的补丁；重复执行最终可能在没有持久行为变化的情况下成功；并且验证器可能只奖励预期行为的一部分。这些可能性并不意味着自我进化是无效的。它们意味着仅凭终点分数无法定位改进所在的位置。

因此，我们提出两个相关的问题。当自我进化提高基准分数时，修订的技能它本身在新的部署和控制任务变化下能否保持行为改进？无论它是否能够做到，进化与评估生命周期中的哪些部分产生了观察到的收益？

我们将证据组织为效用、行为范围和收益归因的诊断。SkillEvolver、SPARK和CoEvoSkills的标准任务评估首先建立完成工件在两次运行中的观察效用。受控任务变化随后揭示不同的效用范围，而工件和轨迹检查则检验在重用过程中保留的决策。这促使下一步诊断：定位工件进化历史中的改进。SkillRevise、SkillHone和SkillOpt的补充研究使用保留测试和生命周期干预来检查轨迹条件下的修订、基于历史的迭代精炼和验证门控编辑。每项研究都针对识别问题的特定部分。

我们的审计遵循从反馈到修订、选择或采纳、冻结工件部署以及测量的一生周期。我们将历史成功与新鲜留存、最终技能效用与实际父级到最终修订效果、相同任务留存与受控迁移区分开来。然后我们对反馈、优化器采样、门控溯源、工件组件和验证器选择进行干预。一次信息完整性检查将直接答案或或然器暴露作为替代解释进行边界约束；它不是本文的组织假设。

最终结果图有意识地比 自我进化有效 或 自我进化仅利用评估者 更有条件。修订工件可以造成真正的行为改进，但展示的效果往往是狭隘的。单次运行或选择增益不必在新鲜部署中存活。有用的轨迹和反馈必须转化为有效的编辑、采纳到持久工件中，并行为上保留。最后，更好的分数可以反映与测量表面的兼容性，而不是更广泛的任务行为。

我们的核心贡献是一个基于主张的生命周期审计：每个报告的增益都与支持它的干预措施以及干预措施允许的最强结论相匹配。我们做出三项贡献：

- **识别框架**。 我们将报告的增益与保持、行为范围、进化与选择路径以及测量依赖性分开。

这是一个仅用于评审目的的匿名提交。严禁传播、引用或公开分享本稿件。如果被接受，版权和出版详情将出现在最终版本中。

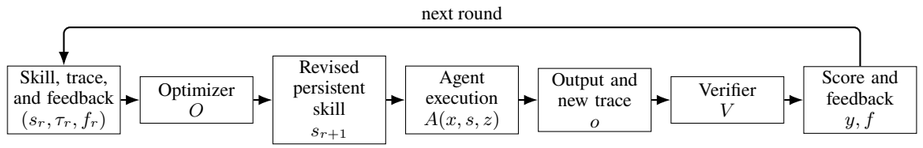


Figure 1: Minimal self-evolution lifecycle. The update operator includes any method-native validation, selection, merge, or adoption before the revised skill is retained.

- **Lifecycle audit.** We apply matched interventions across three representative skill-evolution mechanisms without turning heterogeneous studies into a method leaderboard.
- **Reporting protocol.** We specify the lineage, contrasts, and artifacts required to support each claim level, from selected gain to bounded transfer.

Problem Formulation

Skill Revision and Evaluation

At revision round r, an optimizer updates the current persistent skill artifact:

$$s_{r+1} = O(s_r, \tau_r, f_r), \tag{1}$$

where s_r is the current skill, τ_r is an execution trace, and f_r is feedback that may include a verifier result, a gold answer, a diagnosis, or evaluator feedback. The operator O denotes the method's complete update pathway, including its native merge, validation, selection, or adoption mechanism. A generated candidate that is rejected or never adopted is not s_{r+1}.

The agent then executes task x with the frozen skill:

$$o_r = A(x, s_r, z_r), \tag{2}$$

where z_r represents model sampling, tool use, and environment state, and o_r is the resulting output artifact or state change. A verifier maps the output to a score:

$$y_r = V(x, o_r). \tag{3}$$

The Identification Question

Most evaluations observe:

$$y_{r+1} > y_r. \tag{4}$$

This inequality describes two evaluations; it does not identify what the persistent skill learned. The revised artifact may encode a retained behavioral improvement, the selected score may depend on optimizer or execution variation, or the output may better match the verifier's measurement surface. Infrastructure-invalid observations, such as a verifier that never reaches its semantic assertions, are neither successes nor failure mechanisms and must be repaired or excluded.

Accordingly, we separate two questions:

1. **Effect existence and scope:** Does the revised skill retain a behavioral improvement under fresh deployment, and how far does that effect extend under controlled task variation?
2. **Effect attribution:** Which feedback, revision, selection, artifact, execution, and measurement mechanisms produced the observed score change?

The following experiments address these questions in turn.

Diagnosing the Scope of Skill Utility

The first diagnostic question is where a completed skill is useful. A positive source-task result can arise from content that applies across instances or from a procedure fitted to the original task. We distinguish these possibilities by first examining paired utility in two standard-task runs, then deploying frozen skills under three forms of task variation. Inspection of skill content and execution traces connects the resulting utility profiles to decisions made during deployment.

Experimental Setup

We evaluate SkillEvolver (Zhang et al. 2026a), SPARK (Zhou et al. 2026b), and CoEvoSkills (Zhang et al. 2026b) on SkillsBench (Li et al. 2026), with two runs per method. Each run pairs an execution using the generated or evolved skill with a no-skill execution of the same task. Both runs of each method use the same task set: 81 tasks for SkillEvolver, 84 for SPARK, and 85 for CoEvoSkills.

We report mean verifier reward on a 0–100 scale and the mean within-task difference between the skill and no-skill conditions. For a run with N tasks, the paired gain is

$$\Delta = \frac{100}{N} \sum_{i=1}^{N} \left(r_i^{\text{skill}} - r_i^{\text{no-skill}} \right). \tag{5}$$

We also count tasks with positive, negative, and zero differences.

Source-Task Gains Recur across Runs

All three methods have positive mean paired gains in both runs (Table 1). SkillEvolver gains 25.61 and 25.02 points, and CoEvoSkills gains 10.24 and 9.76 points. For both methods, the aggregate advantage is similar across runs even though absolute rewards change substantially. SkillEvolver's no-skill mean rises from 30.20 to 37.32, while CoEvoSkills' falls from 42.21 to 23.54. A comparison of skill-condition scores alone would obscure this pattern. The within-task contrast locates the observed benefit relative to what the executor achieves without the artifact in each run.

Task-level outcomes clarify how those means arise. SkillEvolver improves 27 tasks and regresses six in run 1, followed by 26 improvements and three regressions in run 2. CoEvoSkills improves 17 tasks in each run, with nine and ten regressions. SPARK has positive gains of 1.84 and 6.45 points, but its first run has nearly balanced improvements and regressions (nine versus eight), whereas the second has 13 improvements and five regressions. A repeated positive

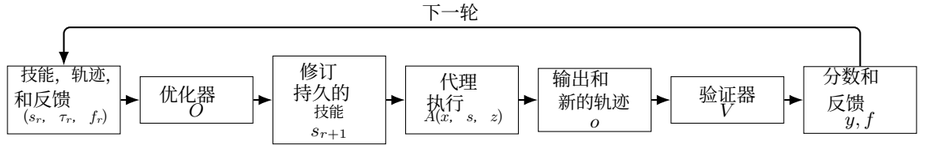


图1：最小自我进化生命周期。更新算子包括修订技能被保留之前的任何方法原生验证、选择、合并或采用。

- **生命周期审计。** 我们在三种代表性的技能进化机制上应用匹配干预，而不会将异质研究变成方法排行榜。
- **报告协议。** 我们指定支持从选定增益到有界迁移的每个主张级别的谱系、对比和工件。

问题公式化

技能修订与评估

在修订轮次 r 中，一个优化器更新当前持久的技能工件：

$$s_{r+1} = O(s_r, \tau_r, f_r), \tag{1}$$

其中 s_r 是当前技能，τ_r 是一个执行轨迹，而 f_r 是可能包含验证器结果、标准答案、诊断或评估器反馈的反馈。运算符 O 表示方法完整的更新路径，包括其原生合并、验证、选择或采用机制。一个被拒绝或从未被采用的生成候选不是 s_{r+1}。

代理随后使用冻结的技能执行任务 x：

$$o_r = A(x, s_r, z_r), \tag{2}$$

其中 z_r 表示模型采样、工具使用和环境状态，而 o_r 是结果输出工件或状态变化。一个验证器将输出映射为一个分数：

$$y_r = V(x, o_r). \tag{3}$$

识别问题

大多数评估观察到：

$$y_{r+1} > y_r. \tag{4}$$

这个不等式描述了两种评估；它并不能识别持久技能学到了什么。修订后的工件可能编码了保留的行为改进，选定的分数可能取决于优化器或执行变化，或者输出可能更好地匹配验证器的测量表面。基础设施无效的观察，例如从未达到其语义断言的验证器，既不是成功机制也不是失败机制，必须被修复或排除。

因此，我们将问题分为两个：

1. **效果存在和范围：** 修订后的技能在全新部署下是否保留了行为改进，以及这种效果在受控任务变化下会延伸多远？
2. **效果归因：** 哪些反馈、修订、选择、工件、执行和测量机制导致了观察到的分数变化？

以下实验依次探讨这些问题。

诊断技能效用的范围

第一个诊断问题是：完成的技能在何处有用。正面的源任务结果可能源于适用于多个实例的内容，也可能源于为原始任务拟合的程序。我们通过首先检查两个标准任务运行中的成对效用，然后以三种任务变化形式部署冻结的技能来区分这些可能性。通过检查技能内容和执行轨迹，可以将产生的效用特征与部署过程中做出的决策联系起来。

实验设置

我们在SkillsBench（Li等人，2026年）上评估SkillEvolver（张等人，2026a）、SPARK（周等人，2026b）和CoEvoSkills（张等人，2026b），每种方法进行两次运行。每次运行将使用生成的或进化的技能的执行与相同任务的技能无关的执行配对。每种方法的两次运行都使用相同的任务集：SkillEvolver有81个任务，SPARK有84个任务，CoEvoSkills有85个任务。

我们报告在0-100的刻度上的平均验证器奖励，以及技能和技能无关条件之间的平均任务差异。对于包含 N 个任务的运行，配对增益是

$$\Delta = \frac{100}{N} \sum_{i=1}^{N} \left(r_i^{\text{skill}} - r_i^{\text{no-skill}} \right). \tag{5}$$

我们还统计了具有正、负和零差异的任务。

源任务增益循环

所有三种方法在两次运行中均表现出正的平均配对增益（表1）。SkillEvolver分别获得25.61和25.02分，CoEvoSkills分别获得10.24和9.76分。对于这两种方法，尽管绝对奖励发生了显著变化，但它们的综合优势在运行之间保持相似。SkillEvolver的无技能均值从30.20上升到37.32，而CoEvoSkills则从42.21下降到23.54。仅比较技能条件分数会掩盖这一模式。任务内对比将观察到的收益定位在执行者每次运行中未使用该工具所能达到的水平上。

任务级结果阐明这些结果如何产生。SkillE-волюция在运行1中改进了27项任务并倒退了六项，随后在运行2中改进了26项并倒退了三项。CoEvoSkills在每次运行中改进了17项任务，分别有九项和十项倒退。SPARK的收益为1.84和6.45分，但其第一次运行中改进和倒退几乎平衡（九项对八项），而第二次运行则有13项改进和五项倒退。一个重复的积极

Method	Run	Tasks	No skill	Skill	Δ	Improved	Regressed	Tied
SkillEvolver	1	81	30.20	55.81	+25.61	27	6	48
SkillEvolver	2	81	37.32	62.34	+25.02	26	3	52
SPARK	1	84	17.82	19.66	+1.84	9	8	67
SPARK	2	84	14.37	20.81	+6.45	13	5	66
CoEvoSkills	1	85	42.21	52.44	+10.24	17	9	59
CoEvoSkills	2	85	23.54	33.31	+9.76	17	10	58

Table 1: Two runs per method on SkillsBench. Scores are mean verifier rewards multiplied by 100; Δ is the mean paired gain in percentage points. Each method uses an identical task set across its two runs.

A. Overall utility on task variants

Method	Variants	No skill	Skill	Δ	Improved	Regressed	Tied
SkillEvolver	222	41.87	53.81	+11.94	47	20	155
SPARK	221	41.61	42.25	+0.65	32	37	152
CoEvoSkills	221	41.61	36.60	−5.01	26	41	154

B. Utility by type of task change

Method	P1: Representation		P2: Instance		P3: Requirements	
	N	Δ	N	Δ	N	Δ
SkillEvolver	77	+11.85	68	+14.56	77	+9.72
SPARK	77	+2.00	68	−5.23	76	+4.53
CoEvoSkills	77	−3.59	68	−9.51	76	−2.42

Table 2: Skill utility under task variation: descriptive summary of historical and recovery evaluations. Panel A reports overall outcomes; Panel B separates representation, instance, and requirement changes. Each method is paired with its own same-variant no-skill baseline. Scores and within-variant differences use the scale in Table 1.

mean can therefore coexist with different concentrations of benefit and harm.

The common finding is a recurring aggregate benefit on the source tasks, with heterogeneous effects across individual tasks. The next diagnostic question is which parts of this usefulness survive task changes. The next comparison holds the source skill fixed and changes the task, so that the executor must reuse existing content rather than receive a skill evolved for the new instance.

Task Changes Separate Different Scopes of Utility

Probing what must change during reuse. We apply each source-task skill to controlled variants without further evolution. Input-representation changes (P1) modify names, column arrangements, or schemas. The executor must recover the same semantic roles from a different presentation. Task-instance changes (P2) replace the data or instance structure, requiring the procedure to operate on new values and relationships. Coupled-requirement changes (P3) alter or add requirements that must be satisfied together, requiring the executor to adapt its solution while preserving the relevant constraints. These probes separate input interpretation, instance-specific computation, and requirement handling; they are not successive difficulty levels.

The complete record contains 222 variants and 886 condition-level results across no skill and the three methods. Table 2 summarizes historical and recovery evaluations for 222 SkillEvolver variants and 221 each for SPARK and CoEvoSkills. Each method's utility is measured against the no-skill result on the same variant. The paired contrast measures utility under the changed task; P1–P3 locate its behavioral scope.

Source-task gains have different extensions. The positive source-task results lead to three distinct variant profiles. SkillEvolver increases mean reward from 41.87 to 53.81, a gain of 11.94 points, with 47 improvements and 20 regressions. Its skill utility extends beyond the instances on which the artifacts were produced. SPARK's average gain is much smaller, at 0.65 points, with 32 improvements and 37 regressions. CoEvoSkills changes from 41.61 to 36.60, a decline of 5.01 points, with 26 improvements and 41 regressions. Thus, a positive result on the original tasks can accompany positive, nearly neutral, or negative average utility after task variation. Source-task success alone does not characterize the scope of a skill.

Instance changes expose the clearest separation. SkillEvolver remains positive under all three probes: 11.85 points on P1, 14.56 on P2, and 9.72 on P3. Improvements outnumber regressions within each category, at 15 versus seven, 15 versus six, and 17 versus seven. Its positive overall result is therefore supported across all three types of change rather than being concentrated in one category. The largest gain occurs on P2, where the source instance has been replaced and the executor must recompute a solution using the new task data.

P2 also produces the largest average decline for the other two methods: 5.23 points for SPARK and 9.51 points for CoEvoSkills. This shared location of their largest decline makes instance changes particularly informative for the di-

方法	Run	任务	无技能	技能	Δ	改进	退化	Tied
SkillEvolver	1	81	30.20	55.81	+25.61	27	6	48
SkillEvolver	2	81	37.32	62.34	+25.02	26	3	52
SPARK	1	84	17.82	19.66	+1.84	9	8	67
SPARK	2	84	14.37	20.81	+6.45	13	5	66
CoEvoSkills	1	85	42.21	52.44	+10.24	17	9	59
CoEvoSkills	2	85	23.54	33.31	+9.76	17	10	58

表1：每个方法在SkillsBench上运行两次。分数是平均验证奖励乘以100； Δ 是平均配对增益（以百分点表示）。每种方法在其两次运行中使用相同的任务集。

A. 在任务变体上的总体效用

方法	变体	无技能	技能	Δ	改进	退化	Tied
SkillEvolver	222	41.87	53.81	+11.94	47	20	155
SPARK	221	41.61	42.25	+0.65	32	37	152
CoEvoSkills	221	41.61	36.60	−5.01	26	41	154

B. 任务变更类型下的效用

方法	P1: 表示		P2: 实例		P3: 要求	
	N	Δ	N	Δ	N	Δ
SkillEvolver	77	+11.85	68	+14.56	77	+9.72
SPARK	77	+2.00	68	−5.23	76	+4.53
CoEvoSkills	77	−3.59	68	−9.51	76	−2.42

表2：任务变化下的技能效用：历史和恢复评估的描述性总结。面板A报告总体结果；面板B将表示、实例和需求变化分开。每种方法都与同变体的无技能基线配对。分数和变体内差异使用表1中的量表。

平均值因此可以与不同浓度的利益和危害共存。

常见发现是源任务上反复出现的综合效益，但在单个任务中存在异质性效应。下一个诊断问题是哪些部分的有效性在任务变化后仍然存在。下一次比较保持源技能固定，改变任务，以便执行者必须重用现有内容，而不是接收为新的实例进化出的技能。

任务变化分离不同效用范围

探测重用时必须改变的内容。 我们将每个源任务技能应用于受控变体而不进行进一步进化。输入表示变化（P1）修改名称、列排列或模式。执行者必须从不同的表示中恢复相同的语义角色。任务实例变化（P2）替换数据或实例结构，要求程序对新值和关系进行操作。耦合需求变化（P3）改变或添加必须一起满足的需求，要求执行者调整其解决方案，同时保留相关约束。这些探测将输入解释、特定于实例的计算和需求处理分开；它们不是连续的难度级别。

完整记录包含 222 个变体和跨越无技能以及三种方法共 886 个条件级结果。表 2 总结了 222 个 SkillEvolver 变体的历史和恢复评估结果，以及 SPARK 和 CoEvoSkills 各 221 个变体的评估结果。每种方法的效用都是与同一变体上的无技能结果进行对比测量的。配对对比测量了在任务变化下的效用；P1–P3 定位其行为范围。

P1–P3 定位其行为范围。

源任务收益具有不同的扩展性。 正面的源任务结果导致三种不同的变体特征。SkillEvolver将平均奖励从41.87提高到53.81，提升了11.94分，其中47次改进和20次退化。其技能效用超越了生成工件所使用的实例。SPARK的平均提升远小，为0.65分，其中32次改进和37次退化。CoEvoSkills从41.61降至36.60，下降了5.01分，其中26次改进和41次退化。因此，在原始任务上的正面结果可能伴随着任务变化后的正面、接近中性或负面的平均效用。源任务的成功并不能表征技能的适用范围。

实例变化暴露了最清晰的区分。 SkillEvolver在所有三种探针下均保持正面：P1上11.85分，P2上14.56分，P3上9.72分。每个类别中改进次数均多于退化次数，分别为15对7、15对6和17对7。因此，其正面总体结果是在三种变化类型中均得到支持，而非集中在某一类别。最大提升发生在P2上，此时源实例已被替换，执行者必须使用新的任务数据重新计算解决方案。

P2 也为其他两种方法产生了最大的平均下降：SPARK 为 5.23 分，CoEvoSkills 为 9.51 分。它们最大下降的共同位置使得实例变化对诊断特别具有信息量。

agnosis. The relevant question is whether a source artifact helps the executor reconstruct the procedure for a new instance, rather than merely repeat an execution that worked before. SkillEvolver shows the strongest positive utility profile on this probe.

Representation and requirements reveal different sensitivities. SPARK gains 2.00 points on P1 and 4.53 on P3 despite its negative P2 result. Its overall mean consequently combines benefits under some changes with losses under another. CoEvoSkills declines on both P1 and P3, by 3.59 and 2.42 points, so its negative overall result extends across the three probes. These profiles explain why a single aggregate variant score is an incomplete description: handling renamed inputs, instantiating a procedure on new data, and satisfying coupled requirements are different demands on skill reuse. The breakdown locates the observed strengths and failures without treating all task changes as interchangeable.

Benefit magnitude and benefit frequency differ. SPARK's positive mean coexists with more regressions than improvements. Reward magnitudes determine the mean, whereas the sign counts describe how often the outcome changes in each direction. Both are needed to interpret utility. Moreover, 155 of 222 SkillEvolver variants, 152 of 221 SPARK variants, and 154 of 221 CoEvoSkills variants tie with their no-skill counterparts. Most evaluations therefore preserve the baseline reward, and the aggregate differences arise from a minority of changed outcomes. A reward tie does not reveal whether the executor ignored the skill or followed a different trajectory to the same score. This motivates inspecting how skill content enters execution, rather than inferring skill use directly from the final reward.

Tracing Utility to Decisions Made during Reuse

The variant profiles establish where skill utility is observed. To examine how that utility is expressed, we inspect the source artifacts and execution traces in the signal-detection and scheduling cases. The analytical unit is a decision that connects stored content to a changed task: recognizing the inputs, choosing or revising an operation, and checking the resulting output. This separates the procedure available in the artifact from the behavior actually produced when the executor applies it.

Diagnostic rules preserve criteria while allowing new solutions. The inspected SkillEvolver signal-detection artifact separates candidate discovery, diagnosis, and local refinement. Its rules specify when a candidate requires further checking and when the search should be adjusted. In the P1 and P3 executions, the executor uses these checks to distinguish competing explanations and revise the search before choosing an output. The reusable element is the criterion for accepting a candidate; the candidate itself and the numerical search settings are recomputed from the changed input.

This distinction matters because a stored action sequence can remain executable while the reason for trusting its result has changed. In the corresponding CoEvoSkills failure traces, executing the period-search routine does not reliably resolve the competing signal interpretations. The contrast directs attention to the checks between operations: whether an intermediate result is accepted, rejected, or used to trigger another search. The table records an outcome difference; the trace shows the decision at which the executions diverge.

The same task family also exposes a boundary of this behavior: both skills fail on the replacement-instance P2 variant. Successful reuse under a changed representation or requirement does not ensure success when the underlying signal changes. This within-family contrast makes the scope question concrete. The diagnostic rules help resolve some changed executions, while the replacement instance remains a failure even with those rules available.

Output constraints preserve the task contract. The scheduling case exposes a complementary form of reusable content. The inspected SkillEvolver artifact states repair constraints relative to the supplied instance and baseline schedule. This permits the concrete schedule to change while preserving the conditions that make the repair valid. The corresponding executions handle changed inputs and requirements while retaining those checks. SkillEvolver and SPARK both succeed on the three inspected scheduling variants, showing that this behavior is not exclusive to a single method.

The CoEvoSkills P1 failure is more localized: the output satisfies other structural and policy checks but violates the required local-minimality condition. The diagnostic value lies in this separation. Producing a largely valid schedule and preserving the full repair contract are different outcomes, even when they arise from similar high-level workflows. Examining the failed predicate connects the reward change to a specific behavioral requirement rather than treating the entire execution as an undifferentiated failure.

From behavioral evidence to lifecycle attribution. Across these cases, successful reuse combines preserved decision criteria with task-dependent execution. Input names, search parameters, and constructed solutions can change, while diagnostic checks and correctness constraints remain relevant. This provides a concrete interpretation of the behavior sought by the variant probes: a useful skill supplies guidance for recomputing a solution, including how to recognize and correct an unsuitable intermediate result. SkillEvolver's positive profile across P1–P3 is consistent with this pattern, and the inspected traces show instances in which the pattern is expressed.

The diagnostic sequence now separates three objects: the utility of a completed artifact, the scope of that utility under task changes, and the decisions observed during its use. Locating an improvement in the evolution process requires an additional contrast: the actual parent artifact against the adopted final artifact, followed by interventions on the feedback, selection, content, or verifier involved. The following studies apply these contrasts to SkillRevise, SkillHone, and SkillOpt. They move from identifying useful behavior to testing whether it is retained and which lifecycle components produce it.

诊断。相关的问题是，一个源工件是否有助于执行者重建新实例的流程，而不仅仅是重复以前奏效的执行。SkillEvolver在这个探测任务上显示出最强的正效用特征。

表征和要求揭示了不同的敏感性。SPARK 在 P1 上获得 2.00 分，在 P3 上获得 4.53 分，尽管其 P2 结果为负。其总体均值因此在一些变化下结合了收益，而在另一些变化下则造成了损失。CoEvoSkills 在 P1 和 P3 上都下降了 3.59 分和 2.42 分，因此其负的总体结果贯穿了三个探针。这些特征解释了为什么单一的汇总变体分数是一种不完整的描述：处理重命名输入、在新数据上实例化程序以及满足耦合要求是对技能重用的不同需求。这种分解定位了观察到的优势和失败，而没有将所有任务变化视为可互换的。

收益幅度和收益频率不同。SPARK 的正向均值与更多的回归重于改进共存。奖励幅度决定均值，而符号计数描述了结果在每个方向上变化的频率。两者都是解释效用所必需的。此外，222 个 SkillEvolver 变体中有 155 个、221 个 SPARK 变体中有 152 个以及 221 个 CoEvoSkills 变体中有 154 个与无技能对照版本持平。因此大多数评估保留了基线奖励，而汇总差异源于少数变化的结果。奖励持平并不能揭示执行者是否忽略了技能或遵循了不同的轨迹达到了相同的分数。这促使我们检查技能内容如何进入执行，而不是直接从最终奖励推断技能使用。

追踪重用过程中的效用决策

Thevariant profiles 建立技能效用观察的位置。为了检验该效用如何表达，我们检查了信号检测和调度案例中的源代码件和执行轨迹。分析单元是连接存储内容到变更任务的决策：识别输入、选择或修订操作、检查结果输出。这区分了源代码件中可用的程序与执行器应用时实际产生的行为。

诊断规则保留标准的同时允许新解决方案。 The inspected SkillEvolver 信号检测源代码件分离了候选发现、诊断和局部细化。其规则指定候选何时需要进一步检查以及何时应调整搜索。在 P1 和 P3 执行中，执行器使用这些检查来区分竞争解释并在选择输出前修订搜索。可重用元素是接受候选的标准；候选本身和数值搜索设置是从变更输入中重新计算的。

这个区别很重要，因为一个存储的动作序列可以在信任其结果的原因发生变化时仍然可执行。在相应的 CoEvoSkills 失败轨迹中，执行周期搜索例程并不能可靠地解决竞争信号解释。对比关注操作之间的检查：中间结果是否被接受、拒绝或用于触发另一个搜索。表格记录了一个结果差异；轨迹显示了执行在哪个决策点分叉。

相同的任务系列也暴露了这种行为的边界：两种技能在替换实例P2变体上都失败。在改变了表示或要求的情况下成功重用，并不保证在底层信号变化时也能成功。这种系列内部对比使范围问题具体化。诊断规则有助于解决一些改变的执行，但即使有这些规则可用，替换实例器仍然是一个失败。

输出约束保留了任务契约。 调度案例暴露了一种可重用内容的互补形式。被检查的SkillEvolver工件声明了相对于提供的实例和基线调度的修复约束。这允许具体调度改变，同时保留使修复有效的条件。相应的执行处理改变的输入和要求，同时保留那些检查。SkillEvolver和SPARK在三个被检查的调度变体上都成功，表明这种行为并非专属于单一方法。

CoEvoSkills P1失败更具局部性：输出满足其他结构和策略检查，但违反了所需的局部最小性条件。诊断价值在于这种分离。生成一个基本有效的调度和保留完整的修复契约是不同的结果，即使它们来自相似的高级工作流程。检查失败的谓词将奖励变化与特定的行为要求联系起来，而不是将整个执行视为无法区分的失败。

从行为证据到生命周期归因。在这些案例中，成功的重用结合了保留的决策标准与任务相关的执行。输入名称、搜索参数和构建的解决方案可以改变，而诊断检查和正确性约束仍然相关。这为变体探针寻求的行为提供了具体解释：一个有用的技能为重新计算解决方案提供指导，包括如何识别和纠正不合适的中间结果。SkillE-volver在P1–P3中的积极表现与此模式一致，检查的轨迹显示了该模式表达的实例。

当前的诊断序列将三个对象分开：完成工件的效用、该效用在任务变化中的范围，以及其使用过程中观察到的决策。定位进化过程中的改进需要额外的对比：实际父工件与采用的最终工件，随后是对反馈、选择、内容或验证器进行的干预。以下研究将这些对比应用于SkillRevise、SkillHone和SkillOpt。它们从识别有用行为转向测试该行为是否被保留以及哪些生命周期组件产生了它。

Panel and contrast	Main result	Supported inference and boundary
Panel A: Retention under fresh deployment		
Historical pass → fresh repeats	12/24 tasks flip within at least one frozen condition; selected-outcome retention is 47.1% for SkillRevise and 55.0% for SkillHone	A selected pass is not a deployment probability; denominators differ and are not a method ranking.
SkillRevise final → no-skill	16/48 vs. 14/48; +4.2 pp, 95% CI $[-10.4, 20.8]$	No clear fresh-utility advantage on this targeted panel; not a parent-to-final revision estimate.
SkillHone final → no-skill	12/48 vs. 14/48; −4.2 pp, 95% CI $[-20.8, 14.6]$	No clear fresh-utility advantage on this targeted panel; not a parent-to-final revision estimate.
Panel B: Revision effects under controlled variations		
Invoice parent → final	$0/12 \rightarrow 12/12$; three new same-contract corpora: $0/9 \rightarrow 9/9$	Stable but narrow same-contract transfer.
Energy parent → final	Original task plus 5% parameter variant: $1/6 \rightarrow 5/6$	Directional parameter transfer in a small stochastic case.
SkillOpt SearchQA parent → final	Independent held-out $9/24 \rightarrow 17/24$; stable-transfer gate fails	Local held-out gain, not stable general procedure learning.

Table 3: Evidence for retained skill effects. Panel A contains 24 targeted tasks, three conditions, and two fresh deployments per condition (144/144 behavior-valid runs). Different historical-retention denominators preclude horizontal method ranking. The no-skill condition calibrates execution and selection variability; final-versus-no-skill does not identify revision effects.

Identifying Retained Skill Effects

Evaluation Design and Contrasts

We analyze three persistent-skill systems with distinct revision mechanisms. SkillRevise converts execution traces and diagnoses into bounded repair principles (Liu et al. 2026b). SkillHone accumulates practice history and iteratively reflects on the persistent skill (Li and Hu 2026). SkillOpt produces structured bounded edits and uses a validation gate and rejected buffer before adoption (Yang et al. 2026). The systems are studied as representative mechanisms, not as directly comparable leaderboard entries.

At the paper level, the evidence is exploratory: individual interventions use frozen inputs and predeclared gates, but the combined studies are designed to identify candidate mechanisms rather than estimate benchmark-wide prevalence. A targeted 24-task SkillsBench panel tests whether historical outcomes survive fresh deployment. The panel is stratified to expose stability failures and method disagreements; it is not a random sample. Three controlled mechanism cases then compare complete parent and final artifacts under predeclared variations. Within each contrast, we hold the executor model, harness, task instance, budget, and verifier version fixed unless one is the named intervention; exact prompts, hashes, seeds, and stopping decisions are preserved in the artifact package. The task is the primary analysis unit, while repeated runs are deployments of the same task-condition pair.

Four comparisons must remain distinct. Historical-pass to fresh-repeat comparisons test retention of a selected outcome, not advantage over a baseline. Final-to-no-skill comparisons estimate fresh utility, not the effect of revision. Only actual-parent to final comparisons identify a revision effect. Applying that comparison to controlled variants tests bounded transfer, not population-average benefit.

Retention under Fresh Deployment

This experiment tests whether historically selected outcomes survive when the final artifact is frozen and redeployed. Holding task, skill, and condition fixed does not make execution deterministic. Across the 24-task panel, 12 tasks change success status between their two repeats in at least one condition. Aggregate fresh success is 14/48 without a skill, 16/48 with the SkillRevise final artifact, and 12/48 with the SkillHone final artifact. Both matched-lift intervals cross zero (Table 3).

These results establish an identification failure in the common reporting practice: cached or selected success cannot be read as stable deployment capability. They do not establish that either revision method is ineffective. The panel is purposive, and it lacks each method's actual parent condition. Its proper conclusion is narrower: on this sample, neither final-versus-no-skill comparison establishes a clear average fresh-deployment advantage, and fresh repeats are necessary before a selected score is called retained.

Revision Effects under Controlled Variations

These mechanism probes test whether the actual parent-to-final revision effect persists under predeclared task changes. In Invoice, the complete final skill repairs all 12 matched deployments and retains the effect on nine deployments drawn from three disjoint corpora that preserve the contract while changing the documents. This is strong evidence of a stable, same-contract revision effect, but not of broad cross-contract generalization.

The Energy case provides weaker but directionally consistent evidence: the complete final artifact improves the original setting and a 5% parameter variant in five of six

面板与对比	主要结果	支持推理和边界
面板 A：新鲜部署下的留存率		
历史通过 → 新鲜重复	12/24 任务至少翻转一种冻结条件；选择结果保留是 SkillRevise 为 47.1%，而 SkillHone 为 55.0%	一个选择通过不是部署概率；分母不同，也不是方法排名。
SkillRevise 最终 → 无技能	16/48 对比 14/48； +4.2pp，95% CI $[-10.4, 20.8]$	在这个目标面板上没有明显的效用优势；不是从父级到最终修订的估计。
SkillHone 最终 →无技能	12/48 与 14/48； −4.2 pp，95% CI $[-20.8, 14.6]$	在此目标面板上，没有明显的效用优势；不是从父版到最终修订的估计。
面板 B：受控变化下的修订效果		
发票父版 →最终	$0/12 \rightarrow 12/12$; 三个新的相同合约语料库: $0/9 \rightarrow 9/9$	稳定但狭窄的相同合约迁移。
能量父 →最终	原始任务加5%参数变体： $1/6 \rightarrow 5/6$	小随机情况下的方向参数迁移。
SkillOpt SearchQA 父 → 最终	独立保留集 $9/24 \rightarrow 17/24$; 稳定迁移门控失败	本地保留集增益，非稳定泛化程序学习。

表 3：保留技能效应的证据。面板 A 包含 24 个目标任务、三种条件和每个条件下的两次新部署（144/144 行为有效运行）。不同的历史保留分母妨碍了横向方法排名。无技能条件校准执行和选择变异性；最终与无技能对比无法识别修订效应。

识别保留技能效应

评估设计与对比

我们分析了三个具有不同修订机制的持久技能系统。SkillRevise将执行轨迹和诊断转化为有界的修复原则（Liu等人，2026b）。SkillHone积累实践历史并迭代地反思持久技能（Li和Hu，2026）。SkillOpt生成结构化的有界编辑，并在采用前使用验证门和拒绝缓冲区（Yang等人，2026）。这些系统作为代表性机制进行研究，而不是作为可直接比较的排行榜条目。

在论文层面，证据是探索性的：单个干预使用冻结输入和预声明门，但综合研究旨在识别候选机制而非估计基准范围内的普遍性。一个目标 24 任务 SkillsBench 面板测试历史结果是否能在新部署中存活。该面板分层以暴露稳定性失败和方法分歧；它不是随机样本。然后比较三种受控机制案例，在预声明变化下对比完整父级和最终工件。在每个对比中，我们保持执行器模型、测试框架、任务实例、预算和验证器版本固定，除非它是命名干预；精确提示、哈希、种子和停止决策保存在工件包中。任务是主要分析单元，而重复运行是相同任务-条件对的部署。

必须保持四个比较的独立性。历史-重复比较测试的是选定结果的保持程度，而非相对于基线的优势。最终-无技能比较估算的是新鲜效用，而非修订的影响。只有实际父代-最终比较能识别修订效果。将此比较应用于受控变体则测试的是有限迁移，而非群体平均收益。

新部署下的保持率

该实验测试了历史选择的结果在最终工件冻结并重新部署时是否仍然存活。保持任务、技能和条件不变并不能使执行结果确定。在24项任务面板中，12项任务在至少一个条件下两次重复的成功状态发生变化。没有技能的总体新成功率为48/14，使用SkillRevise最终工件的总体新成功率为48/16，使用SkillHone最终工件的总体新成功率为48/12。两个匹配提升间隔都穿过零（表3）。

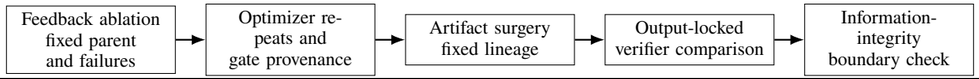
这些结果在常见报告实践中确立了一个识别失败：缓存的或选择的成功不能被视为稳定的部署能力。它们没有确立任何一种修订方法都无效。该面板是经过深思熟虑的，并且缺少每种方法的实际父条件。其恰当的结论更为狭窄：在这个样本中，无论是最终工件与无技能的比较，都没有明确地表明平均新部署优势，并且在新重复之前，选择的分数不能被称为保持。

R受控变量下的修订效果

变化

这些机制测试了在实际父级到最终修订效果下，是否在预定义任务变化时仍然存在。在发票中，完整的最终技能修复了所有12个匹配的部署，并保留了从三个保持合同但更改文档的独立语料库中抽取的九个部署的效果。这是稳定、同一合同修订效果的强有力证据，但不是跨合同泛化的证据。

TheEnergy案例提供了较弱但方向上一致的证据：最终完整成果改进了原始设置，且在六次部署中有五次的5%参数变体



Lifecycle stage	SkillRevise	SkillHone	SkillOpt
Feedback and revision	Diagnosis ablation; revision traces	Feedback-view ablation	Feedback branches; optimizer repeats
Selection and adoption	Revision history	Iterative honing history	Native validation gate; accepted and rejected candidates
Artifact mediation	Not yet matched at the same strength	Invoice and Energy surgery	SearchQA component surgery
Measurement	No output-locked comparison yet	Radar verifier swap	Spreadsheet dual scoring
Information integrity	Artifact forensic	Artifact forensic	Wrong-metadata and oracle controls

Figure 2: Lifecycle attribution design and current evidence coverage. The upper panel locates each intervention; the lower panel records method coverage. Populated cells do not imply equivalent strength, and missing cells bound cross-method claims.

deployments, compared with one of six for the parent. The sample is too small to support a prevalence or generality claim. In SkillOpt SearchQA, the final artifact improves independent held-out accuracy from 9/24 to 17/24, yet fails the predeclared stability threshold. It therefore supports local held-out benefit rather than stable transferable procedure learning.

Taken together, revised skill artifacts can produce retained behavioral effects, but a selected benchmark pass alone does not identify such an effect, and the effects established so far remain narrow rather than broadly generalizable.

Attributing Observed Skill Gains

Lifecycle Interventions

The experiments in this section intervene at different lifecycle stages. They do not estimate one common effect size. Each intervention changes one object while holding its relevant lineage fixed: feedback-view studies hold the parent and failures fixed; optimizer-repeat studies hold the input evidence fixed; gate audits preserve accepted and rejected candidates; artifact surgery holds the surrounding artifact fixed; and verifier counterfactuals hold the output fixed.

Figure 2 makes the current coverage and its gaps explicit. The shared audit logic is that feedback access, candidate revision, validation acceptance, adoption into the persistent artifact, and retained behavioral improvement are distinct events.

Revision and Selection Pathways

These interventions test whether the observed endpoint depends on the feedback view, optimizer sample, or selection gate. In SkillRevise, the complete three-revision pipeline succeeds on 53/86 tasks, compared with 28/86 when diagnosis is removed. Task-level discordance is 26 gains versus one loss, and 2,326 of 3,648 revision entries explicitly use verifier evidence. This demonstrates that diagnosis changes the selected endpoint and that the revision process is strongly evaluator-facing. It does not, by itself, establish which changes are retained under fresh deployment.

SkillHone separates access to feedback from stable benefit. Detailed sanitized, binary, and redacted-tail feedback obtain first-deployment success on 4/12, 4/12, and 5/12 cases, while four of 18 repeated cells flip. Feedback can trigger revisions, but these data do not support a stable advantage for one feedback view.

SkillOpt further separates candidate production from attribution and adoption. In matched SearchQA optimizer repeats, raw-gold and outcome-only views each produce four applied edits. Yet the preregistered between-condition effect size is only $D = .044 < .20$, despite weak lexical decodability, so a single patch cannot be cleanly attributed to feedback view. Across outcome-only and trajectory-only branches, all 6/6 candidates change and 3/6 pass the gate, but every acceptance is driven by the same Blackwater validation item. In OfficeQA, a selected one-item gain disappears in fresh matched deployment. Thus edit production, gate acceptance, and retained improvement cannot be collapsed into one event.

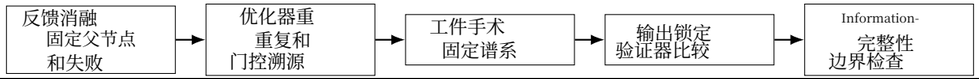
Artifact-Level Mediation

This experiment tests whether a feedback-derived component, rather than unrelated edits in the full artifact, causes the behavior after adoption. Let P denote the complete parent, F the complete final artifact, $P + B$ the parent with component B, and $F - B$ the final artifact with that component removed.

These interventions establish that an adopted artifact component can cause behavior. They do not show that the same mechanism is prevalent across methods. In particular, Skill-Revise lacks a matched surgery of equal strength, and the current cases should be read as existence and boundary evidence.

Measurement Dependence under Fixed Outputs

This counterfactual tests whether a verdict changes when the output is fixed and only the measurement changes. In the Radar family, all 12 reconstructed outputs reproduce their historical original-verifier outcomes. Of six original passes, only one remains a pass under a reference-calibrated strict verifier; the five valid flips arise from specification-implied



生命周期阶段	SkillRevise	SkillHone	SkillOpt
反馈与修订	诊断消融；修订轨迹	反馈视图消融	反馈分支；优化器
选择与采纳	修订历史	迭代打磨历史	重复 本地验证关卡；已接受以及被拒绝的候选者
文物调解	尚未在相同	发票和能源手术	SearchQA组件手术
测量	力量 尚未有输出锁定比较	雷达验证器交换	电子表格双重评分
信息完整性	文物取证	文物取证	错误元数据和预言机控制

图2：生命周期归因设计及当前证据覆盖范围。上层面板定位每个干预措施；下层面板记录方法覆盖范围。填充的单元格不表示同等强度，缺失的单元格界定跨方法主张。

与父案例的六分之一相比。样本量太小，无法支持普遍性或泛化性声明。在SkillOpt SearchQA中，最终成果将独立验证准确率从9/24提升至17/24，但未能通过预设的稳定性阈值。因此它支持局部验证集收益而非稳定的可迁移过程学习。

综合来看，修订后的技能工件可以产生持久的 behavioral effects，但仅通过选择的基准测试并不能识别这种效果，且迄今为止建立的效果仍然比较狭窄，而非广泛可推广。

归因于观察到的技能提升

生命周期干预

本节的实验在生命周期的不同阶段进行干预。它们不估计一个共同的效应量。每次干预改变一个对象，同时保持其相关谱系固定：反馈-视图研究固定父对象和失败；优化器-重复研究固定输入证据；门控审计保留接受和拒绝的候选；工件手术固定周围工件；验证器反事实研究固定输出。

图2使当前的覆盖范围及其空白显式化。共享的审计逻辑是：反馈访问、候选修订、验证接受、采用到持久工件以及持久的 behavioral improvement 是不同的独立事件。

修订和选择路径

这些干预措施测试了观察到的终点是否取决于反馈视图、优化器样本或选择门。在SkillRevise中，完整的三次修订流程在53/86个任务上成功，而移除诊断时仅为28/86。任务级别的分歧为26个收益与1个损失，且在3,648个修订条目中，有2,326个明确使用了验证器证据。这表明诊断改变了选定的终点，且修订过程强烈依赖于评估器。它本身并不能确定哪些变更在全新部署下被保留。

SkillHone将访问反馈与稳定收益分离。详细的清理、二进制和截断尾部反馈在4/12、4/12和5/12个案例中实现了首次部署成功，而18个重复单元中有4个发生了翻转。反馈可以触发修订，但这些数据不支持任何一种反馈视图具有稳定优势。

SkillOpt 进一步将候选生成与归因和采纳分开。在匹配的 SearchQA 优化器重复中，原始金标和仅结果视图各产生四个应用过的编辑。然而，预先注册的跨条件效应量仅为 $D = .044 < .20$，尽管词汇解码能力较弱，因此单个补丁无法清晰地归因于反馈视图。在仅结果和仅轨迹分支中，所有 6/6 候选都发生变化，且 3/6 通过了门控，但每个接受都是由同一个 Blackwater 验证项驱动的。在 OfficeQA 中，一个被选中的一项增益在新的匹配部署中消失。因此，编辑生成、门控接受和保留的改进不能合并为一个事件。

Artifact-Level Mediation

该实验测试反馈派生的组件，而不是完整工件中的无关编辑，是否导致采纳后的行为。令 P 表示完整父项，F 表示完整最终工件，$P+B$ 表示包含组件 B 的父项，以及 $F-B$ 表示移除该组件的最终工件。

这些干预措施表明，一个采用的组件可以导致行为。它们并未表明相同的机制在所有方法中都普遍存在。特别是，Skill-Revise缺乏同等强度的匹配手术，当前案例应被视为存在和边界证据。

固定输出下的测量依赖性

此反事实测试在输出固定且仅测量变化时，裁决是否会改变。在Radar系列中，所有12个重建输出都重现了其历史原始-验证器结果。在六个原始通过中，只有一个在参考校准的严格验证器下仍然通过；五个有效翻转源于规范隐含的检查。

Case	Result	Maximum claim
Invoice	$P = 0/12$, $P + B = 12/12$, $F - B = 0/12$, $F = 12/12$	B is necessary and sufficient for the narrow same-contract effect.
Energy	Local mediation in the original task; necessity and sufficiency do not survive the variant	Local component effect, not transferable mediation.
SearchQA	9 repairs, 0 regressions; stable success 7/12, below the strong gate	Directional mediation on part of the held-out set.

Table 4: Artifact-level mediation evidence.

checks. The same behavior can therefore receive different valid verdicts because the measurement surfaces differ.

Measurement dependence can also operate in the other direction. For identical Spreadsheet workbook outputs, the reported value-based evaluator scores the complete SkillOpt artifact at 10/12, whereas joint value-and-formula recalculation scores it at 8/12. Representation, cache materialization, or formula inspection can therefore change what the evaluator observes. This dual scoring is not identical in design to the Radar output-locked swap, but it reinforces the narrower point that score is a property of output plus measurement, not behavior alone.

Neither result proves semantic failure, exploitation, or benchmark-wide prevalence. To identify evolution-induced verifier dependence, one would additionally need matched no-skill, parent, and final outputs under both original and calibrated verifiers, and test whether the parent-to-final verifier gap increases.

Information-Integrity Boundary

This supporting check tests whether restricted answer or oracle information enters and controls the artifact. High-confidence direct-answer or oracle matches are absent from 86 SkillRevise and 85 SkillHone natural artifacts. SkillOpt conditions containing the feedback-derived answer-surface section follow incorrect metadata in 0/15 probes. In contrast, an unsafe hidden-oracle positive control changes probe success from 0/5 to 5/5 and leaves private-oracle or gold markers.

These checks show that the audit can detect direct exposure when it is deliberately introduced, while the natural artifacts do not support direct copying as a universal explanation. They do not prove the absence of implicit answer-following policies or all forms of leakage.

Observed score gains can arise at multiple lifecycle stages: feedback shapes candidate revisions, selection can amplify narrow improvements, adopted artifact components can mediate real behavior, and valid verifiers can disagree on fixed outputs. These mechanisms must be identified separately before a score gain is interpreted as skill learning.

Audit step	Required test → maximum supported claim
Validity	Execution and verifier status plus assertion coverage → eligible observation
Retention	Frozen actual-parent/final fresh repeats → retained artifact effect
Behavioral scope	Entity, parameter, input, contract, or family variants → bounded controlled transfer
Evolution pathway	Feedback ablation, optimizer repeats, and gate provenance → pathway dependence or selection mediation
Measurement	Output-locked calibrated verifier comparison → measurement dependence
Information integrity	Provenance, wrong-metadata, and positive control → exploitation or confirmed leakage, when supported

Table 5: Evidence requirements and claim boundaries. The rows answer complementary questions rather than forming a scalar risk ladder.

From Reported Gains to Auditable Claims

Evidence Requirements and Claim Boundaries

The audit is not a scalar ladder from *safe* to *unsafe*. Its steps answer independent questions and impose distinct claim boundaries.

A selected gain is weaker than a retained artifact effect; a retained effect on one instance is weaker than controlled transfer. Evolution-pathway and measurement claims remain meaningful even when retention is not established. Conversely, verifier-aware language or evaluator dependence must not be relabeled as leakage without restricted-information provenance and behavioral evidence.

Lifecycle Reporting Protocol

An auditable self-evolution report should preserve the following minimum evidence:

- **Before optimization:** parent artifact, task and split identity, failure pool, environment, and verifier version.
- **During revision:** optimizer-visible traces and feedback, candidate patches, optimizer samples, prompts, and usage.
- **At selection and adoption:** accepted and rejected artifacts, gate items, per-item decisions, and exact adopted artifact.
- **At deployment:** frozen artifact, actual-parent and no-skill conditions, fresh repeats, controlled variants, and output bytes.
- **At scoring:** verifier logs, assertion coverage, reference calibration, and any alternative verifier applied to the same output.
- **For cost:** separate optimizer, executor, verifier, and invalid-attempt ledgers.

The complete artifact appendix should additionally record hashes, commands, seeds, environment revisions, timeouts, and stopping decisions. This protocol turns a post-evolution score into an auditable lineage and limits the paper's language to the strongest matched contrast actually performed.

Case	结果	最大主张
发票	$P = 0/12$, $P + B = 12/12$, $F - B = 0/12$, $F = 12/12$	B 是必要且充分的对于狭义相同合约效应
能量	局部中介在原始任务中必要性和充分性不会存活变异体	局部组件效应，不是可转移的中介
SearchQA	9次修复，0回归；稳定成功7/12，低于强门限	部分定向中介保留集。

表4：工件级中介证据。

因此，相同的行为可能会因为测量表面不同而获得不同的有效裁决。

测量依赖性也可以反向运作。对于相同的电子表格工作簿输出，基于值的报告评估器将完整的 SkillOpt 工件评为 10/12，而联合值和公式重新计算将其评为 8/12。因此，表示、缓存材料化或公式检查可以改变评估器所观察到的结果。这种双重评分在设计中并不等同于雷达输出锁定交换，但它加强了更狭窄的观点：分数是输出和测量的属性，而不仅仅是行为。

这两个结果都无法证明语义失败、利用或基准的普遍性。要识别进化引起的验证器依赖性，还需要在原始和校准的验证器下匹配无技能、父代和最终输出，并测试父代到最终验证器差距是否增加。

信息完整性边界

此辅助检查测试受限答案或预言机信息是否进入并控制工件。高置信度的直接答案或预言机匹配在 86 个 SkillRevise 和 85 个 SkillHone 自然工件中不存在。包含反馈派生答案表面部分的 SkillOpt 条件在 0/15 次探测中遵循错误的元数据。相比之下，一个不安全的隐藏预言机阳性对照将探测成功率从 0/5 改变为 5/5，并保留私有预言机或金标记。

这些检查表明，审计可以在直接暴露被故意引入时检测到，而自然产生的伪影并不支持直接复制作为普遍解释。它们并不能证明不存在隐式答案跟随策略或所有形式的泄露。

观察到的分数提升可能出现在多个生命周期阶段：反馈会塑造候选修订，选择可以放大狭窄的改进，已采用的工件组件可以调解实际行为，而有效的验证器可能对固定输出存在分歧。在将分数提升解释为技能学习之前，必须单独识别这些机制。

审计步骤	必需测试 → 最大支持的声明
有效性	执行和验证器状态加上断言覆盖率 → 符合资格观察
<code><redacted></code>	冻结实际父节点/最终新鲜重复 → 保留的工件效果
行为	实体、参数、输入、合约或家族变体 → 有界
范围	受控迁移
进化	反馈消融、优化器重复以及门控溯源 →
路径	路径依赖或选择介导
测量	输出锁定校准验证器比较 →测量依赖
信息完整性	来源、错误元数据和正向控制 → 利用或确认泄露，当受支持时

表 5：证据要求与声明边界。行回答的是互补性问题，而不是形成标量风险阶梯。

从报告收益到可审计声明证据要求与声明边界

审计并非从安全到不安全的标量阶梯。它的步骤回答独立的问题，并施加不同的声明边界。

一个选定的收益弱于一个保留的工件效应；一个实例上的保留效应弱于受控迁移。即使保留未建立，进化路径声明和测量声明仍然是有意义的。相反，验证器感知语言或评估器依赖不得在没有受限信息来源和行为证据的情况下被重新标记为泄漏。

生命周期报告协议

可审计的自进化报告应保留以下最低证据：

- **优化前：** 父工件、任务和分割身份、失败池、环境以及验证器版本。
- **在修订期间：** 优化器可见轨迹和反馈、候选补丁、优化器样本、提示和使用情况。
- **在选择和采纳时：** 接受和拒绝的工件、门控项、单项决策以及精确采纳的工件。
- **在部署时：** 冻结的工件、实际父和技能无条件、新鲜重复、受控变体和输出字节。
- **在评分时：** 验证器日志、断言覆盖率、参考校准以及应用于相同输出的任何替代验证器。
- **关于成本：** 分离的优化器、执行器、验证器和无效尝试账本。

完整的工件附录还应记录哈希值、命令、种子、环境修订、超时和停止决策。该协议将进化后的分数转化为可审计的谱系，并将论文的语言限制为实际执行的最强匹配对比。

Related Work

Skill Evolution and Empirical Evaluation

Agent-skill research spans generation, revision, selection, retrieval, organization, and reinforcement learning. Skills-Bench and SWE-Skills-Bench compare skill and no-skill utility across diverse or software-engineering tasks (Li et al. 2026; Han et al. 2026). SkillLearnBench evaluates continual skill generation (Zhong et al. 2026), while SkillRevise, SkillHone, SkillOpt, SkillRL, and CoEvoSkills propose different revision or learning loops (Liu et al. 2026b; Li and Hu 2026; Yang et al. 2026; Xia et al. 2026; Zhang et al. 2026b). Recent empirical work broadens the object of study: From Raw Experience to Skill Consumption separates experience generation, extraction, and consumption (Huang et al. 2026); How Well Do Agentic Skills Work in the Wild studies realistic skill retrieval (Liu et al. 2026a); SkillCoach evaluates process-level skill use (Zhu et al. 2026); and SkillJuror isolates the effect of skill organization (Chen et al. 2026).

These studies show that outcome-only evaluation is already recognized as incomplete. Our focus is narrower and complementary: when a persistent skill is revised and the post-revision score rises, what evidence identifies a retained artifact effect rather than a selected or measurement-dependent endpoint? We therefore organize evidence by lifecycle contrast rather than by method score.

Evaluation and Attribution of Agent Improvement

Agent evaluation increasingly examines process quality, sequential adaptation, verifier reliability, and counterfactual behavior. Counterfactual Trace Auditing measures how a provided skill changes an agent trajectory (Zhou et al. 2026a). SEA-Eval evaluates self-evolving agents beyond isolated episodes (Jiang et al. 2026). Co-evolving verifier work and grader audits study interactions between the policy and its measurement signal (Zhang et al. 2026b,c). More broadly, reward hacking and test-set contamination research demonstrates that performance on a proxy or exposed test need not imply the intended capability (Amodei et al. 2016; Oren et al. 2023).

We combine these concerns within the lifecycle of persistent skill revision. The central distinction is between retention, behavioral scope, evolution-and-selection pathway, and output-level measurement dependence. Information-integrity checks play a supporting role: they bound direct answer or oracle exposure, but do not convert every verifier-aware adaptation into a leakage claim.

Limitations

Our evidence is mechanism-focused rather than a benchmark-wide prevalence study. The 24-task retention panel is purposive and contains only two fresh deployments per condition; it exposes instability but cannot estimate deployment rates. The controlled Invoice, Energy, SearchQA, Radar, Spreadsheet, and OfficeQA probes use small, heterogeneous task families rather than a fully crossed method-by-intervention design. SkillRevise, for example, lacks matched artifact surgery and output-locked measurement evidence of the same strength, while the strongest verifier swap is concentrated in the Radar family. We therefore do not claim that any observed mechanism is universal or that one method outperforms another.

Several conclusions are also bounded by observability. Output-locked comparisons identify measurement dependence but do not determine which verifier best captures intended behavior without an external specification. Artifact forensics and wrong-metadata controls can rule out simple direct exposure, but not implicit answer-following policies or every form of contamination. Finally, our scope is frozen-agent, feedback-driven systems with persistent external artifacts; parameter-updating self-evolution, multimodal skills, and online environments may exhibit different lifecycle dynamics. Confirmatory evidence will require a shared sampling plan that fully crosses actual parent and final artifacts, fresh repeats, controlled variants, and calibrated verifiers.

Conclusion

We diagnose where the gains from self-evolving persistent skills reside and how far they extend. All three methods in the standard-task evaluations have positive paired gains in both runs, but task variation separates their utility profiles. SkillEvolver remains positive across representation, instance, and requirement changes; SPARK combines gains on two probes with a decline on instance changes; CoEvoSkills declines on all three. The inspected traces identify decision criteria and correctness checks expressed during successful reuse. Complementary lifecycle interventions show that adopted artifact components can change behavior, selected outcomes can disappear under fresh deployment, and fixed outputs can receive different verdicts under different verifiers. Together, these findings distinguish useful artifacts, their behavioral scope, and the processes that produce or measure their apparent improvement.

A higher benchmark score should therefore be reported only at the strongest level supported by matched evidence: a selected gain, a retained artifact effect, bounded controlled transfer, evolution-pathway dependence, or measurement dependence. We provide a lifecycle audit that links each claim to the required intervention and reporting evidence. Claims of transferable skill learning require frozen parent-to-final improvement that survives fresh deployment and predeclared task variation under valid measurement; weaker evidence should be reported more narrowly.

References

Amodei, D.; Olah, C.; Steinhardt, J.; Christiano, P.; Schulman, J.; and Mané, D. 2016. Concrete Problems in AI Safety. arXiv:1606.06565.

Chen, Z.; Guo, Z.; Huang, B.; Lu, B.; Lin, J.; Zhou, Y.; and Zhang, W. 2026. SkillJuror: Measuring How Agent Skill Organization Changes Runtime Behavior. arXiv:2606.11543.

Han, T.; Zhang, Y.; Song, W.; Fang, C.; Chen, Z.; Sun, Y.; and Hu, L. 2026. SWE-Skills-Bench: Do Agent Skills Actually Help in Real-World Software Engineering? arXiv:2603.15401.

Huang, Z.; Xu, J.; Yang, Y.; Gong, Z.; Yang, Q.; Tian, M.; Wang, X.; Lv, C.; Gao, X.; Dai, Q.; Liu, B.; Qiu, K.; Yang, X.; Chen, D.; Zheng, X.; and Luo, C. 2026. From Raw

相关工作

技能进化和实证评估

代理技能研究涵盖生成、修订、选择、检索、组织和强化学习。Skills-Bench 和 SWE-Skills-Bench 在多样或软件工程任务中比较技能与无技能的效用（Li 等人，2026；Han 等人，2026）。SkillLearnBench 评估持续技能生成（Zhong 等人，2026），而 SkillRevise、SkillHone、SkillOpt、SkillRL 和 CoEvoSkills 提出不同的修订或学习循环（Liu 等人，2026b；Li 和 Hu，2026；Yang 等人，2026；Xia 等人，2026；Zhang 等人，2026b）。最近的实证工作扩展了研究对象：从原始经验到技能消耗将经验生成、提取和消耗分离（Huang 等人，2026）；《代理技能在现实中的表现如何》研究现实技能检索（Liu 等人，2026a）；SkillCoach 评估过程级技能使用（Zhu 等人，2026）；SkillJuror 隔离技能组织的影响（Chen 等人，2026）。

这些研究表明，仅基于结果的评估已被认为是不完整的。我们的关注点更狭窄且具有补充性：当持久的技能被修订且修订后的分数上升时，哪些证据可以识别出保留的工件效应而不是被选中的或测量依赖的终点？因此，我们按生命周期对比来组织证据，而不是按方法分数来组织。

代理改进的评估与归因

代理评估越来越多地考察过程质量、序列适应、验证器可靠性和反事实行为。反事实轨迹审计测量提供的技能如何改变代理轨迹（Zhou 等人，2026a）。SEA-Eval 超越孤立场景评估自进化代理（Jiang 等人，2026）。共进化验证器工作和评分器审计研究策略与其测量信号之间的交互（Zhang 等人，2026b,c）。更广泛地说，奖励破解和测试集污染研究证明，在代理或暴露的测试上的表现不一定意味着预期的能力（Amodei 等人，2016；Oren 等人，2023）。

我们将这些关注点结合在持久技能修订的生命周期中。核心区别在于持久性、行为范围、进化与选择路径，以及输出级测量依赖性。信息完整性检查起辅助作用：它们限制直接答案或预言机暴露，但不会将所有验证器感知的适应性转换成泄漏声明。

局限性

我们的证据是机制导向的，而非全局普遍性研究。24任务保持性面板是经过精心设计的，每个条件下仅包含两次新鲜部署；它暴露了不稳定性，但无法估计部署率。受控的Invoice、Energy、SearchQA、Radar、Spreadsheet和OfficeQA探测任务使用小型异构任务族，而非完全交叉的方法-干预设计。例如，SkillRevise缺乏同等强度的工件手术和输出锁定测量证据，而最强的验证器交换集中在Radar族。因此，我们不声称任何观察到的机制是普遍的，也不声称一种方法优于另一种方法。

一些结论也受限于可观察性。输出锁定比较能识别测量依赖性，但无法确定哪个验证器最能捕捉预期行为（除非有外部规范）。工件取证和错误元数据控制可以排除简单的直接暴露，但无法排除隐式答案跟随策略或任何形式的污染。最后，我们的范围是冻结代理、反馈驱动的系统（具有持久外部工件）；参数更新自进化、多模态技能和在线环境可能表现出不同的生命周期动态。确认性证据将需要一个共享的采样计划，该计划完全跨越实际父代和最终工件，包括新鲜重复、受控变体和校准验证器。

结论

我们诊断了自演化持久技能的收益所在及其影响范围。在标准任务评估中，所有三种方法在两次运行中都显示出正向配对收益，但任务变化区分了它们的效用特征。SkillE-volver在表示、实例和需求变化中始终保持正值；SPARK在两个探针上结合收益，但在实例变化中收益下降；CoEvoSkills在所有三种情况下都收益下降。检查的轨迹识别了在成功重用时表达的决策标准和正确性检查。互补的生命周期干预表明，采用的工件组件可以改变行为，选定的结果在全新部署下可能消失，固定的输出在不同验证器下可能获得不同判定。综合这些发现，可以区分有用工件、其行为范围以及产生或衡量其明显改进的过程。

因此，只有在最强有力的匹配证据支持时，才应报告更高的基准分数：一个选定的收益、一个保留的工件效应、有界的受控迁移、进化路径依赖或测量依赖。我们提供了一个生命周期审计，将每个主张与所需的干预和报告证据联系起来。可迁移技能学习的主张需要冻结的父到最终改进，该改进在全新部署和预声明的任务变化下在有效测量中幸存；较弱证据应更严格地报告。

参考文献

阿莫迪，D.；奥拉赫，C.；斯坦哈特，J.；克里斯蒂亚诺，P.；舒尔曼，J.；以及马内，D. 2016. 人工智能安全的实际问题. arXiv:1606.06565. 陈，Z.；郭，Z.；黄，B.；陆，B.；林，J.；周，Y.；以及张，W. 2026. 技能评审员：衡量代理技能组织如何改变运行时行为. arXiv:2606.11543. 韩 T.；张，Y.；宋，W.；方，C.；陈，Z.；孙，Y.；以及胡，L. 2026. SWE-Skills-Bench：代理技能是否真的有助于实际软件工程？arXiv:2603.15401. 黄，Z.；徐，J.；杨，Y.；龚，Z.；杨，Q.；田，M.；王，X.；吕，C.；高，X.；戴，Q.；刘，B.；邱，K.；杨，X.；陈，D.；郑，X.；以及罗，C. 2026. 从原始

Experience to Skill Consumption: A Systematic Study of Model-Generated Agent Skills. arXiv:2605.23899.

Jiang, S.; Ma, L.; Hong, Z.; Wang, K.; Lu, Z.; Wang, T.; Chen, S.; Zhang, J.; Pan, T.; Li, W.; Liang, J.; and Xiao, Y. 2026. SEA-Eval: A Benchmark for Evaluating Self-Evolving Agents Beyond Episodic Assessment. arXiv:2604.08988.

Li, X.; Liu, Y.; Chen, W.; You, B.; Di, Z.; He, Y.; Zheng, S.; Choe, K. W.; Sun, J.; Wang, S.; Tao, C.; Li, B.; Zhao, X.; Geng, H.; Wu, X.; Zhou, J.; Chen, X.; Xing, H.; Li, Y.; Zeng, Q.; Wang, D.; Wang, Y.; Chaim, R. B.; Jiang, P.; Shen, H.; Kong, L.; Liu, X.; Wang, R.; Liu, X.; Li, J.; Lan, X.; Lin, Y.; Ye, W.; He, J.; Li, S.; Zhang, Y.; Gao, Y.; Li, Y.; Ma, Z.; Jing, L.; Wang, T.; Li, K.; Xue, Y.; Lyu, H.; He, Y.; Tian, Y.; Wu, S.; Wang, B.; Gao, Y.; Chen, B.; Liu, L.; Cheng, S.; Bao, J.; Tong, S.; Xu, S.; Zhuo, T. Y.; Ye, T.; Qi, Q.; Li, M.; Liao, L.; Tan, Z.; Shi, C.; Tang, X.; Tankasala, S.; Yuan, B.; Qian, Y.; Tu, J.; Wang, C.; Sun, Y.; Wang, W.; Taylor, A.; Yang, Z.; Guan, C.; Dong, Z.; Zhang, X.; Dillmann, S.; chung Lee, H.; and Song, D. 2026. SkillsBench: Benchmarking How Well Agent Skills Work Across Diverse Tasks. arXiv:2602.12670.

Li, Z.; and Hu, Y. 2026. SkillHone: A Harness for Continual Agent Skill Evolution Through Persistent Decision History. arXiv:2606.08671.

Liu, Y.; Ji, J.; An, L.; Jaakkola, T.; Zhang, Y.; and Chang, S. 2026a. How Well Do Agentic Skills Work in the Wild: Benchmarking LLM Skill Usage in Realistic Settings. arXiv:2604.04323.

Liu, Y.; Su, Z.; Xie, L.; Zhang, Y.; Zong, Q.; Guo, J.; Xie, Z.; Ji, Y.; Yim, Y.; Luo, H.; Ren, X.; Ruan, C.; Li, H.; and Song, Y. 2026b. SkillRevise: Improving LLM-Authored Agent Skills via Trace-Conditioned Skill Revision. arXiv:2606.01139.

Oren, Y.; Meister, N.; Chatterji, N.; Ladhak, F.; and Hashimoto, T. B. 2023. Proving Test Set Contamination in Black Box Language Models. arXiv:2310.17623.

Xia, P.; Chen, J.; Wang, H.; Liu, J.; Zeng, K.; Wang, Y.; Han, S.; Zhou, Y.; Zhao, X.; Chen, H.; Zheng, Z.; Xie, C.; and Yao, H. 2026. SkillRL: Evolving Agents via Recursive Skill-Augmented Reinforcement Learning. arXiv:2602.08234.

Yang, Y.; Gong, Z.; Huang, W.; Yang, Q.; Zhou, Z.; Huang, Z.; Li, Y.; Gao, X.; Dai, Q.; Liu, B.; Qiu, K.; Yang, Y.; Chen, D.; Yang, X.; and Luo, C. 2026. SkillOpt: Executive Strategy for Self-Evolving Agent Skills. arXiv:2605.23904.

Zhang, G.; Zhu, E.; Zhou, J.; Jia, C.; and Wang, H. 2026a. SkillEvolver: Skill Learning as a Meta-Skill. arXiv:2605.10500.

Zhang, H.; Fan, S.; Zou, H. P.; Chen, Y.; Wang, Z.; Zhou, J.; Li, C.; Huang, W.-C.; Yao, Y.; Zheng, K.; Liu, X.; Li, X.; and Yu, P. S. 2026b. CoEvoSkills: Self-Evolving Agent Skills via Co-Evolutionary Verification. arXiv:2604.01687.

Zhang, X.; Wang, G.; Cui, Y.; Li, Z.; Qiu, W.; Zhu, B.; and He, P. 2026c. Who Grades the Grader? Co-Evolving Evaluation Metrics and Skills for Self-Improving LLM Agents. arXiv:2607.12790.

Zhong, S.; Lu, Y.; Ning, J.; Wan, Y.; Feng, L.; Ao, Y.; Ribeiro, L. F. R.; Dreyer, M.; Ammirati, S.; and Xiong, C. 2026. SkillLearnBench: Benchmarking Continual Learning Methods for Agent Skill Generation on Real-World Tasks. arXiv:2604.20087.

Zhou, X.; Liu, J.; Li, L.; Rossi, R. A.; and Hu, X. 2026a. Counterfactual Trace Auditing of LLM Agent Skills. arXiv:2605.11946.

Zhou, Y.; Dong, Z.; Wang, Z.; Jin, C.; Zhao, S.; Guo, B.; Gu, D.; Zhang, L.; Zhou, M.; and Metaxas, D. N. 2026b. Evidence Over Plans: Online Trajectory Verification for Skill Distillation. arXiv:2605.09192.

Zhu, J.; Mao, K.; Guo, Y.; He, D.; Xu, S.; Gu, S.; and Yue, Y. 2026. SkillCoach: Self-Evolving Rubrics for Evaluating and Enhancing Agentic Skill-Use. arXiv:2607.01874.

经验到技能消耗：对模型生成代理技能的系统研究. arXiv:2605.23899. 姜S.；马，L.；红，Z.；王，K.；陆，Z.；王，T.；陈，S.；张，J.；潘，T.；李，W.；梁，J.；以及萧，Y. 2026. SEA-Eval：一个用于评估自我进化代理的基准，超越情景评估. arXiv:2604.08988. 李，X.；刘，Y.；陈，W.；尤，B.；狄，Z.；何，Y.；郑，S.；赵，K. W.；孙，J.；王，S.；陶，C.；李，B.；赵，X.；耿，H.；吴，X.；周，J.；陈，X.；行，H.；李，Y.；曾，Q.；王，D.；王，Y.；蔡姆，R. B.；姜，P.；沈，H.；孔，L.；刘，X.；王，R.；刘，X.；李，J.；兰，X.；林，Y.；叶，W.；何，J.；李，S.；张，Y.；高，Y.；李，Y.；马，Z.；景，L.；王，T.；李，K.；薛，Y.；吕，H.；何，Y.；田，Y.；吴，S.；王，B.；高，Y.；陈，B.；刘，L.；程，S.；鲍，J.；童，S.；徐，S.；卓，T. Y.；叶，T.；齐，Q.；李，M.；廖，L.；谭，Z.；石，C.；唐，X.；坦卡萨拉，S.；袁，B.；钱，Y.；涂，J.；王，C.；孙，Y.；王，W.；泰勒，A.；杨，Z.；关，C.；董，Z.；张，X.；迪尔曼，S.；李，H. chung；以及宋，D. 2026. 技能基准：评估代理技能在不同任务中表现如何. arXiv:2602.12670. 李，Z.；以及胡，Y. 2026. 技能打磨：通过持久决策历史持续进化代理技能的框架. arXiv:2606.08671. 刘，Y.；纪，J.；安，L.；雅各卡洛，T.；张，Y.；以及长，S. 2026a. 代理技能在现实世界中表现如何：基准测试真实场景中的LLM技能使用情况. arXiv:2604.04323. 刘，Y.；苏，Z.；谢，L.；张，Y.；宗，Q.；郭，J.；谢，Z.；纪，Y.；英，Y.；罗，H.；任，X.；阮，C.；李，H.；以及宋，Y. 2026b. 技能修订：通过轨迹条件技能修订改进LLM编写的代理技能. arXiv:2606.01139. 奥伦，Y.；梅斯特，N.；查特吉，N.；拉达克，F.；以及哈希莫托，T. B. 2023. 证明黑盒语言模型的测试集污染. arXiv:2310.17623. 夏，P.；陈，J.；王，H.；刘，J.；曾，K.；王，Y.；韩，S.；周，Y.；赵，X.；陈，H.；郑，Z.；谢，C.；以及姚，H. 2026. 技能RL：通过递归技能增强强化学习进化代理. arXiv:2602.08234. 杨，Y.；龚，Z.；黄，W.；杨，Q.；周，Z.；黄，Z.；李，Y.；高，X.；戴，Q.；刘，B.；邱，K.；杨，Y.；陈，D.；杨，X.；以及罗，C. 2026. 技能优化：自我进化代理技能的执行策略. arXiv:2605.23904. 张，G.；朱，E.；周，J.；贾，C.；以及王，H. 2026a. 技能进化器：技能学习作为一种元技能. arXiv:2605.10500. 张，H.；范，S.；邹，H. P.；陈，Y.；王，Z.；周，J.；李，C.；黄，W.-C.；姚，Y.；郑，K.；刘，X.；李，X.；以及余，P. S. 2026b. CoEvoSkills：通过协同进化验证自我进化代理技能. arXiv:2604.01687. 张，X.；王，G.；崔，Y.；李，Z.；邱，W.；朱，B.；以及何，P. 2026c. 谁评审评审者？协同进化评估指标和技能，用于自我改进的LLM代理. arXiv:2607.12790. 钟，S.；陆，Y.；宁，J.；万，Y.；冯，L.；敖，Y.；里贝罗，L. F. R.；德雷尔，M.；阿米拉蒂，S.；以及熊，C.

2026. SkillLearnBench：对真实世界任务上代理技能生成的持续学习方法进行基准测试。arXiv:2604.20087. 周翔；刘佳；李丽；Rossi, R. A.；和胡翔。2026a. LLM代理技能的逆向轨迹审计。arXiv:2605.11946. 周宇；董哲；王铮；金晨；赵思；郭斌；郭东；张立；周明；和Metaxas, D. N. 2026b. 证据胜于计划：技能蒸馏的在线轨迹验证。arXiv:2605.09192. 朱杰；毛坤；郭勇；何丹；徐思；郭松；和岳宇。2026. SkillCoach：用于评估和增强代理技能使用的自进化评分标准。arXiv:2607.01874。